# Project Assignment

AIT 203: Optimization

**Question 2-a**

**Energy-Aware Elastic Cloud Resource Allocation Using Penalty Method with Integer-Promoting Regularization**

**Team Members:**
BT2024212 Aryan Sharma
BT2024028 Sutaria Parth Anandkumar

# 1 Introduction

Cloud platforms must allocate heterogeneous Virtual Machine (VM) types such that CPU, memory, I/O, storage, and network demands are satisfied while minimizing cost. Classical allocation models use linear objectives, but real-world systems exhibit nonlinear energy consumption and require integer quantities of VMs. To capture these characteristics, we extend the standard model by incorporating:

- nonlinear energy consumption (power-law scaling),
- need for integer-like VM quantities,
- hard resource constraints,
- supply limits per VM type.

To handle these simultaneously, we construct a smooth, differentiable objective and apply the Penalty Method for constrained optimization.

# 2 Problem Setup

We consider $j = 1, \ldots, 20$ VM types.

## 2.1 Decision Variables

$$x_j \geq 0 \qquad \text{allocation units of VM type } j.$$

### 2.2 Parameters

For each VM type $j$:

$$c_j: \text{ cost}, \quad p_j: \text{energy coefficient}, \quad a_{j,k}: \text{resource contribution}, \quad u_j: \text{upper supply limit}.$$

Workload demands:

$$D_k \quad \text{for } k \in \{\text{CPU, RAM, NET, STOR, IO}\}.$$

# 3 Objective Function

The complete smooth objective is:

$$f(x) = \underbrace{\sum_{j=1}^{20} c_j x_j}_{\text{Linear Cost}} + \underbrace{\eta \sum_{j=1}^{20} p_j x_j^{1.4}}_{\text{Nonlinear Energy}} + \underbrace{\lambda \sum_{j=1}^{20} (1 - \cos(2\pi x_j))}_{\text{Integer Regularizer}}$$
$$+ \underbrace{P \sum_k \max\left(0,\ D_k - \sum_{j=1}^{20} a_{j,k} x_j\right)^2}_{\text{Constraint Penalty}}. \tag{1}$$

- **Linear Cost:** Each VM type has a fixed cost $c_j$ per unit. Increasing $x_j$ increases total monetary cost proportionally.
- **Nonlinear Energy Cost:** Energy usage grows with a power-law exponent 1.4, reflecting realistic CPU power scaling. This term penalizes extremely large VM allocations more heavily than small ones.
- **Integer-Promoting Regularizer:** The smooth term $1-\cos(2\pi x_j)$ is minimized when $x_j$ is an integer. It encourages near-integer allocations while keeping the objective differentiable.
- **Constraint Penalty:** Any shortfall between provided resources and required demand is penalized quadratically. The penalty weight $P$ grows during optimization, driving the solution toward feasibility.

# 4 Constraints

$$\sum_{j=1}^{20} a_{j,k} x_j \geq D_k, \qquad \forall k, \tag{2}$$

$$0 \leq x_j \leq u_j, \qquad \forall j. \tag{3}$$

# 5 Gradients

$$\nabla_{x_j} C_{\text{linear}} = c_j,$$

$$\nabla_{x_j} C_{\text{energy}} = 1.4\,\eta\, p_j\, x_j^{0.4},$$

$$\nabla_{x_j} C_{\text{int}} = 2\pi\lambda \sin(2\pi x_j),$$

Define

$$s_k = \max(0,\ D_k - A_k x).$$

Then

$$\nabla_x C_{\text{penalty}} = -\,2P\, A^\top s.$$

# 6 Penalty Method Algorithm

1. Initialize $x^{(0)}$, $P = P_0$, $\lambda = \lambda_0$.
2. Repeat:
   (a) Gradient descent update:
   $$x \leftarrow \Pi_{[0,u]}\left(x - \alpha \nabla f(x)\right),$$
   (b) Update penalty:
   $$P \leftarrow \gamma_P P, \qquad \lambda \leftarrow \gamma_\lambda \lambda.$$
3. Stop when $\max_k s_k < \varepsilon$.

Option A used in this experiment keeps $\lambda$ fixed and increases $P$.

# 7 Numerical Results

## 7.1 Dataset

Twenty synthetic VM types were generated following realistic scaling patterns for cost, energy coefficients, and resources. Demand:

$$D = (50,\ 180,\ 15,\ 800,\ 40).$$

## 7.2 Solver Configuration

- Gradient descent per outer iteration: 2000 steps
- Initial penalty: $P_0 = 1$
- Penalty growth factor: $\beta = 5$
- Tolerance: $\varepsilon = 10^{-10}$

## 7.3 Final Results

Final objective (penalized): 181.56

## 7.4 Final Allocation Table

| | VM Type | x_j (allocation) | Rounded x_j | Cost c_j | Energy p_j | CPU | RAM | NET | STOR | IO | Upper Bound u_j |
|---|---|---|---|---|---|---|---|---|---|---|---|
| **0** | 1 | 6.956073 | 7 | 3.792752 | 0.728379 | 1.000000 | 3.247059 | 0.535931 | 21.095943 | 1.484700 | 18 |
| **1** | 2 | 3.830999 | 4 | 4.229819 | 0.781137 | 1.403422 | 4.401140 | 0.691043 | 20.000000 | 1.534318 | 15 |
| **2** | 3 | 0.866942 | 1 | 5.038737 | 0.771130 | 1.354524 | 4.819909 | 0.698061 | 33.890589 | 2.145724 | 13 |
| **3** | 4 | 0.004104 | 0 | 4.629686 | 0.725972 | 1.016386 | 3.881416 | 0.832769 | 27.965601 | 2.410520 | 7 |
| **4** | 5 | 3.839528 | 4 | 5.080708 | 0.820820 | 1.923230 | 6.054588 | 0.848149 | 42.265846 | 2.074807 | 17 |
| **5** | 6 | 2.888742 | 3 | 4.680184 | 0.811244 | 1.503903 | 5.901492 | 0.780473 | 35.936285 | 3.432756 | 15 |
| **6** | 7 | 5.517554 | 6 | 4.836725 | 0.756728 | 1.283574 | 7.214089 | 0.975351 | 51.611361 | 3.249333 | 18 |
| **7** | 8 | 0.013759 | 0 | 5.858942 | 0.871293 | 2.984624 | 8.454383 | 1.476902 | 39.208566 | 4.394099 | 5 |
| **8** | 9 | 0.032713 | 0 | 5.937329 | 0.869235 | 2.376712 | 9.318282 | 1.385929 | 61.745298 | 3.994338 | 13 |
| **9** | 10 | 0.012334 | 0 | 5.891274 | 0.817847 | 3.622935 | 8.099580 | 1.482457 | 64.249037 | 5.223466 | 7 |
| **10** | 11 | 0.026442 | 0 | 6.156627 | 0.840587 | 3.130887 | 9.387884 | 1.671013 | 58.689674 | 4.530798 | 8 |
| **11** | 12 | 0.053438 | 0 | 6.154265 | 0.833730 | 2.910147 | 10.258507 | 1.545060 | 67.192789 | 4.217161 | 14 |
| **12** | 13 | 3.018229 | 3 | 6.295477 | 0.835407 | 3.644959 | 11.189400 | 1.864177 | 77.588355 | 5.471322 | 19 |
| **13** | 14 | 0.096947 | 0 | 6.763747 | 0.926775 | 4.394668 | 12.409555 | 2.000639 | 74.431075 | 6.003039 | 8 |
| **14** | 15 | 0.067723 | 0 | 6.617430 | 0.924633 | 3.646865 | 11.369302 | 1.929752 | 80.354768 | 5.638582 | 13 |
| **15** | 16 | 0.125902 | 0 | 7.180754 | 0.892272 | 4.273120 | 13.713643 | 1.967181 | 73.528349 | 6.076251 | 15 |
| **16** | 17 | 0.137522 | 0 | 7.877680 | 0.928050 | 4.876351 | 14.530884 | 2.204720 | 93.005538 | 6.414086 | 19 |
| **17** | 18 | 1.037826 | 1 | 7.623033 | 0.968424 | 5.074731 | 13.902880 | 2.194954 | 100.762044 | 6.597389 | 19 |
| **18** | 19 | 0.123626 | 0 | 7.984528 | 0.929411 | 5.485694 | 14.780354 | 2.525434 | 96.573506 | 7.312312 | 14 |
| **19** | 20 | 0.086151 | 0 | 7.897466 | 0.911102 | 4.960874 | 13.636881 | 2.626425 | 94.735967 | 7.501206 | 17 |

Figure 1: Final VM Allocation Table

### 7.5 Alloted vs Demand Chart

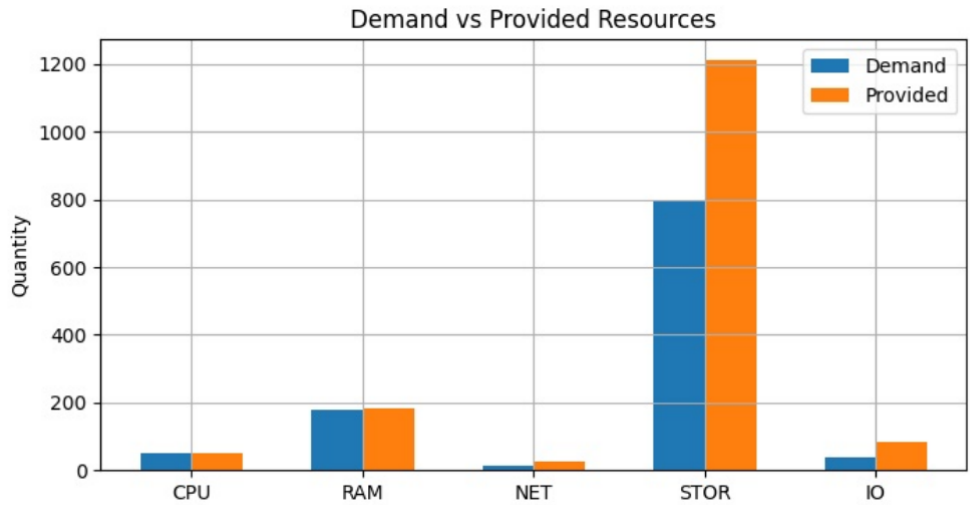



Figure 2: Convergence Behavior of the Penalized Objective

### 7.6 Convergence Plot

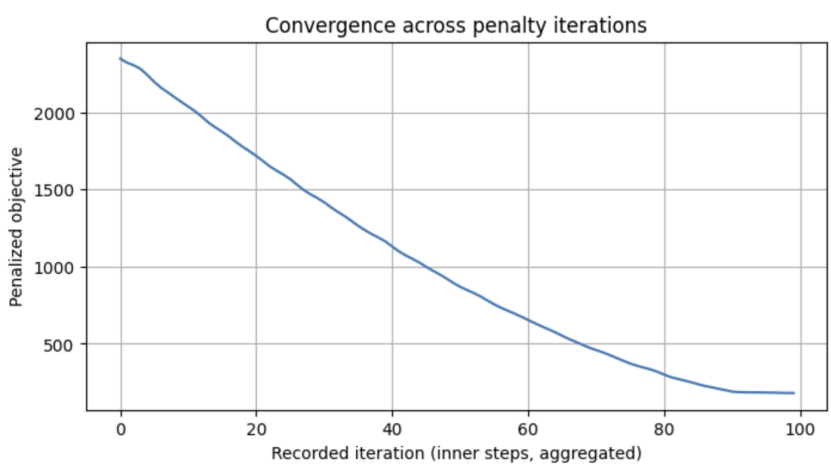



Figure 3: Convergence Behavior of the Penalized Objective

## 8 Interpretation of Results

### 8.1 Behavior of the Solution

- The feasibility constraint was satisfied immediately, indicating strong resource capacity.
- Integer regularization nudged allocations close to integer values without destabilizing optimization.

- The nonlinear energy term discouraged large allocations of extremely powerful VMs.

### 8.2 Trade-offs Observed

- Higher-CPU VMs with balanced RAM-to-cost ratios were selected.
- The quadratic penalty ensured feasibility but remained numerically stable.
- The integer term adds slight bias toward integral solutions but does not dominate the cost landscape.

## 9 Conclusion

Using the full objective specified in the project description and the Penalty Method algorithm, we obtained a feasible, integer-like VM allocation that minimizes combined cost and energy while satisfying all workload requirements. The approach is smooth, differentiable, and scalable for practical cloud resource management.

In [1]:
```python
# Importing numpy and pandas
import numpy as np
import pandas as pd
import time
```

In [2]:
```python
# Generating Demo Data

np.random.seed(86)

# Number of VMs
num_vms = 20
num_resources = 5  # CPU, RAM, NET, STORAGE, IO

# Baseline Power of the VM
vm_power = np.linspace(0.5, 2.5, num_vms) + np.random.normal(0, 0.1, num_vms)

# Cost proportional to power of the VM but with some variance
c = 3 + 2.0 * vm_power + np.random.normal(0, 0.2, num_vms)
c = np.clip(c, 2, None)


# Energy score
p = 0.7 + 0.1 * vm_power + np.random.normal(0, 0.03, num_vms)
p = np.clip(p, 0.5, 1.2)


# Resource matrix
A = np.zeros((num_vms, num_resources))

# CPU score
A[:, 0] = 2 * vm_power + np.random.normal(0, 0.3, num_vms)
A[:, 0] = np.clip(A[:, 0], 1, 8)

# RAM score
A[:, 1] = 6 * vm_power + np.random.normal(0, 1.0, num_vms)
A[:, 1] = np.clip(A[:, 1], 2, 32)

# Network score
A[:, 2] = 1.0 * vm_power + np.random.normal(0, 0.1, num_vms)
A[:, 2] = np.clip(A[:, 2], 0.5, 5)

# Storage score
A[:, 3] = 40 * vm_power + np.random.normal(0, 5, num_vms)
A[:, 3] = np.clip(A[:, 3], 20, 200)

# IO throughput score
A[:, 4] = 3 * vm_power + np.random.normal(0, 0.5, num_vms)
A[:, 4] = np.clip(A[:, 4], 1, 10)



# Smaller/cheaper VMs tend to be available in larger quantities
u = np.random.randint(5, 20, num_vms)


# Demand
D = np.array([
    50,   # CPU cores
    180,  # RAM
    15,   # Network
    800,  # Storage
    40    # IO
])




eta = 1.0
P = 5.0      # penalty
lam = 0.5    # integer-pull rate


# Final Params generated
pd.DataFrame({
    "Cost c_j": c,
    "Energy p_j": p,
    "CPU": A[:, 0],
    "RAM": A[:, 1],
    "NET": A[:, 2],
    "STOR": A[:, 3],
    "IO": A[:, 4],
```

```
    "Upper Bound": u
})
```

Out[2]:

| | Cost c_j | Energy p_j | CPU | RAM | NET | STOR | IO | Upper Bound |
|---|---|---|---|---|---|---|---|---|
| **0** | 3.792752 | 0.728379 | 1.000000 | 3.247059 | 0.535931 | 21.095943 | 1.484700 | 18 |
| **1** | 4.229819 | 0.781137 | 1.403422 | 4.401140 | 0.691043 | 20.000000 | 1.534318 | 15 |
| **2** | 5.038737 | 0.771130 | 1.354524 | 4.819909 | 0.698061 | 33.890589 | 2.145724 | 13 |
| **3** | 4.629686 | 0.725972 | 1.016386 | 3.881416 | 0.832769 | 27.965601 | 2.410520 | 7 |
| **4** | 5.080708 | 0.820820 | 1.923230 | 6.054588 | 0.848149 | 42.265846 | 2.074807 | 17 |
| **5** | 4.680184 | 0.811244 | 1.503903 | 5.901492 | 0.780473 | 35.936285 | 3.432756 | 15 |
| **6** | 4.836725 | 0.756728 | 1.283574 | 7.214089 | 0.975351 | 51.611361 | 3.249333 | 18 |
| **7** | 5.858942 | 0.871293 | 2.984624 | 8.454383 | 1.476902 | 39.208566 | 4.394099 | 5 |
| **8** | 5.937329 | 0.869235 | 2.376712 | 9.318282 | 1.385929 | 61.745298 | 3.994338 | 13 |
| **9** | 5.891274 | 0.817847 | 3.622935 | 8.099580 | 1.482457 | 64.249037 | 5.223466 | 7 |
| **10** | 6.156627 | 0.840587 | 3.130887 | 9.387884 | 1.671013 | 58.689674 | 4.530798 | 8 |
| **11** | 6.154265 | 0.833730 | 2.910147 | 10.258507 | 1.545060 | 67.192789 | 4.217161 | 14 |
| **12** | 6.295477 | 0.835407 | 3.644959 | 11.189400 | 1.864177 | 77.588355 | 5.471322 | 19 |
| **13** | 6.763747 | 0.926775 | 4.394668 | 12.409555 | 2.000639 | 74.431075 | 6.003039 | 8 |
| **14** | 6.617430 | 0.924633 | 3.646865 | 11.369302 | 1.929752 | 80.354768 | 5.638582 | 13 |
| **15** | 7.180754 | 0.892272 | 4.273120 | 13.713643 | 1.967181 | 73.528349 | 6.076251 | 15 |
| **16** | 7.877680 | 0.928050 | 4.876351 | 14.530884 | 2.204720 | 93.005538 | 6.414086 | 19 |
| **17** | 7.623033 | 0.968424 | 5.074731 | 13.902880 | 2.194954 | 100.762044 | 6.597389 | 19 |
| **18** | 7.984528 | 0.929411 | 5.485694 | 14.780354 | 2.525434 | 96.573506 | 7.312312 | 14 |
| **19** | 7.897466 | 0.911102 | 4.960874 | 13.636881 | 2.626425 | 94.735967 | 7.501206 | 17 |

In [3]:

```
def C_linear(x):
    return np.sum(c * x)  # linear VM cost

def C_energy(x):
    return eta * np.sum(p * x**1.4)  # nonlinear energy term

def C_int(x):
    return lam * np.sum(1 - np.cos(2 * np.pi * x))  # integer-promotion

def constraint_shortfall(x):
    return np.maximum(0, D - A.T @ x)  # unmet demand

def C_penalty(x):
    s = constraint_shortfall(x)
    return P * np.sum(s**2)  # constraint penalty

def f(x):
    return C_linear(x) + C_energy(x) + C_int(x) + C_penalty(x)
```

In [4]:

```
def grad(x):
    g = np.zeros_like(x)

    # Gradient of linear cost is just the cost vector
    g += c

    # Gradient of nonlinear energy term
    g += 1.4 * eta * p * (x**0.4)

    # Gradient of the integer-promoting penalty
    g += 2 * np.pi * lam * np.sin(2 * np.pi * x)

    # Constraint penalty gradient
    s = constraint_shortfall(x)
    g -= 2 * P * (A @ s)

    return g
```

In [5]:

```
# Ensuring that x remains within valid bounds:
def projection(x):
    return np.clip(x, 0, u)
```

In [6]:
```
P0 = 1.0      # initial penalty
beta = 5.0    # scaling factor for penalty
max_outer = 1000
eps = 1e-10

def penalized_objective(x, P):
    return C_linear(x) + C_energy(x) + C_int(x) + P * np.sum(constraint_shortfall(x)**2)

def penalized_grad(x, P):
    g = c + 1.4 * eta * p * (x**0.4)
    g += 2 * np.pi * lam * np.sin(2 * np.pi * x)
    s = constraint_shortfall(x)
    g -= 2 * P * (A @ s)
    return g

def solve_penalized(P, x_init=None, alpha=0.001, iters=2000, record_every=20):
    x = np.zeros(num_vms) if x_init is None else x_init.copy()
    history = []
    for it in range(iters):
        x = x - alpha * penalized_grad(x, P)
        x = np.clip(x, 0, u)
        if it % record_every == 0:
            history.append(penalized_objective(x, P))
    return x, history



P = P0
x = np.zeros(num_vms)
convergence = []   # flattened inner histories for plotting
penalty_levels = []  # track P at outer iterations
outer_iter = 0

for outer in range(max_outer):
    outer_iter = outer
    x, inner_hist = solve_penalized(P, x_init=x, alpha=0.001, iters=2000, record_every=20)
    convergence.extend(inner_hist)
    penalty_levels.append(P)

    s = constraint_shortfall(x)
    if np.all(s < eps):
        break
    P *= beta

final_P = P
final_x = x
final_shortfall = constraint_shortfall(final_x)
final_obj = penalized_objective(final_x, final_P)
```

In [7]:
```
# Tabular summary of final allocations and VM specs
results = pd.DataFrame({
    "VM Type": np.arange(1, num_vms + 1),
    "x_j (allocation)": final_x,
    "Rounded x_j": np.round(final_x).astype(int),
    "Cost c_j": c,
    "Energy p_j": p,
    "CPU": A[:, 0],
    "RAM": A[:, 1],
    "NET": A[:, 2],
    "STOR": A[:, 3],
    "IO": A[:, 4],
    "Upper Bound u_j": u
})


print("Outer iterations run:", outer_iter + 1)
print("Final penalty P:", final_P)
print("Final objective (penalized):", final_obj)
print("Max shortfall per resource:", final_shortfall.max())
results
```

```
Outer iterations run: 1
Final penalty P: 1.0
Final objective (penalized): 181.56863373843206
Max shortfall per resource: 0.0
```

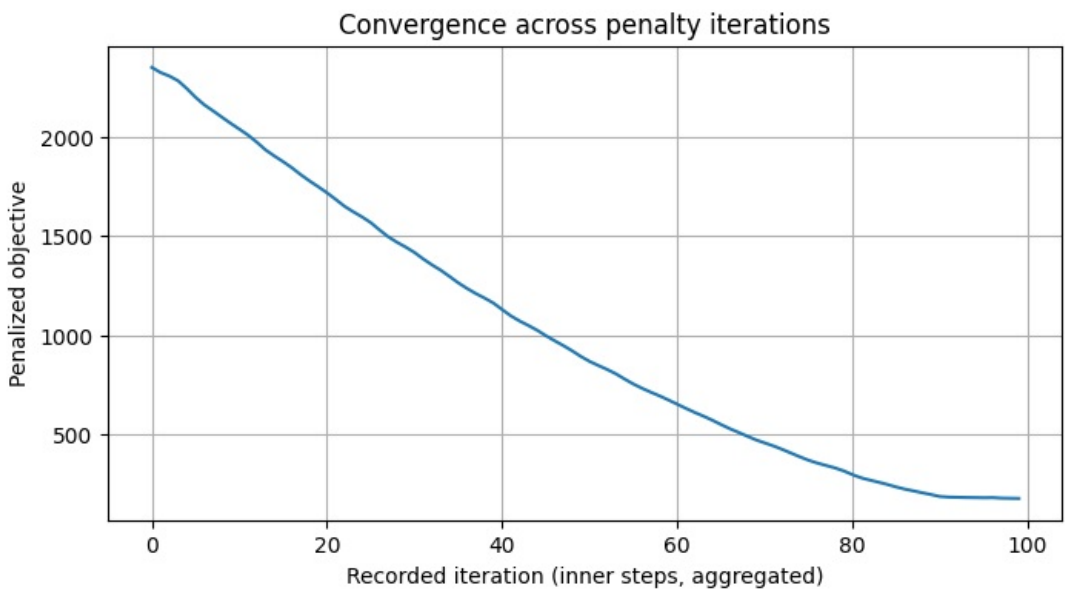
Out[7]:

| | VM Type | x_j (allocation) | Rounded x_j | Cost c_j | Energy p_j | CPU | RAM | NET | STOR | IO | Upper Bound u_j |
|---|---|---|---|---|---|---|---|---|---|---|---|
| **0** | 1 | 6.957287 | 7 | 3.792752 | 0.728379 | 1.000000 | 3.247059 | 0.535931 | 21.095943 | 1.484700 | 18 |
| **1** | 2 | 3.832784 | 4 | 4.229819 | 0.781137 | 1.403422 | 4.401140 | 0.691043 | 20.000000 | 1.534318 | 15 |
| **2** | 3 | 0.868920 | 1 | 5.038737 | 0.771130 | 1.354524 | 4.819909 | 0.698061 | 33.890589 | 2.145724 | 13 |
| **3** | 4 | 0.006210 | 0 | 4.629686 | 0.725972 | 1.016386 | 3.881416 | 0.832769 | 27.965601 | 2.410520 | 7 |
| **4** | 5 | 3.841976 | 4 | 5.080708 | 0.820820 | 1.923230 | 6.054588 | 0.848149 | 42.265846 | 2.074807 | 17 |
| **5** | 6 | 2.891166 | 3 | 4.680184 | 0.811244 | 1.503903 | 5.901492 | 0.780473 | 35.936285 | 3.432756 | 15 |
| **6** | 7 | 5.517820 | 6 | 4.836725 | 0.756728 | 1.283574 | 7.214089 | 0.975351 | 51.611361 | 3.249333 | 18 |
| **7** | 8 | 0.017752 | 0 | 5.858942 | 0.871293 | 2.984624 | 8.454383 | 1.476902 | 39.208566 | 4.394099 | 5 |
| **8** | 9 | 0.036590 | 0 | 5.937329 | 0.869235 | 2.376712 | 9.318282 | 1.385929 | 61.745298 | 3.994338 | 13 |
| **9** | 10 | 0.016729 | 0 | 5.891274 | 0.817847 | 3.622935 | 8.099580 | 1.482457 | 64.249037 | 5.223466 | 7 |
| **10** | 11 | 0.030350 | 0 | 6.156627 | 0.840587 | 3.130887 | 9.387884 | 1.671013 | 58.689674 | 4.530798 | 8 |
| **11** | 12 | 0.057698 | 0 | 6.154265 | 0.833730 | 2.910147 | 10.258507 | 1.545060 | 67.192789 | 4.217161 | 14 |
| **12** | 13 | 3.022852 | 3 | 6.295477 | 0.835407 | 3.644959 | 11.189400 | 1.864177 | 77.588355 | 5.471322 | 19 |
| **13** | 14 | 0.102085 | 0 | 6.763747 | 0.926775 | 4.394668 | 12.409555 | 2.000639 | 74.431075 | 6.003039 | 8 |
| **14** | 15 | 0.072441 | 0 | 6.617430 | 0.924633 | 3.646865 | 11.369302 | 1.929752 | 80.354768 | 5.638582 | 13 |
| **15** | 16 | 0.131562 | 0 | 7.180754 | 0.892272 | 4.273120 | 13.713643 | 1.967181 | 73.528349 | 6.076251 | 15 |
| **16** | 17 | 0.143459 | 0 | 7.877680 | 0.928050 | 4.876351 | 14.530884 | 2.204720 | 93.005538 | 6.414086 | 19 |
| **17** | 18 | 1.043563 | 1 | 7.623033 | 0.968424 | 5.074731 | 13.902880 | 2.194954 | 100.762044 | 6.597389 | 19 |
| **18** | 19 | 0.129730 | 0 | 7.984528 | 0.929411 | 5.485694 | 14.780354 | 2.525434 | 96.573506 | 7.312312 | 14 |
| **19** | 20 | 0.091803 | 0 | 7.897466 | 0.911102 | 4.960874 | 13.636881 | 2.626425 | 94.735967 | 7.501206 | 17 |

In [8]:

```
# Plot: convergence of objective value across inner iterations
import matplotlib.pyplot as plt

plt.figure(figsize=(8,4))
plt.plot(convergence)
plt.xlabel("Recorded iteration (inner steps, aggregated)")
plt.ylabel("Penalized objective")
plt.title("Convergence across penalty iterations")
plt.grid(True)
plt.show()
```

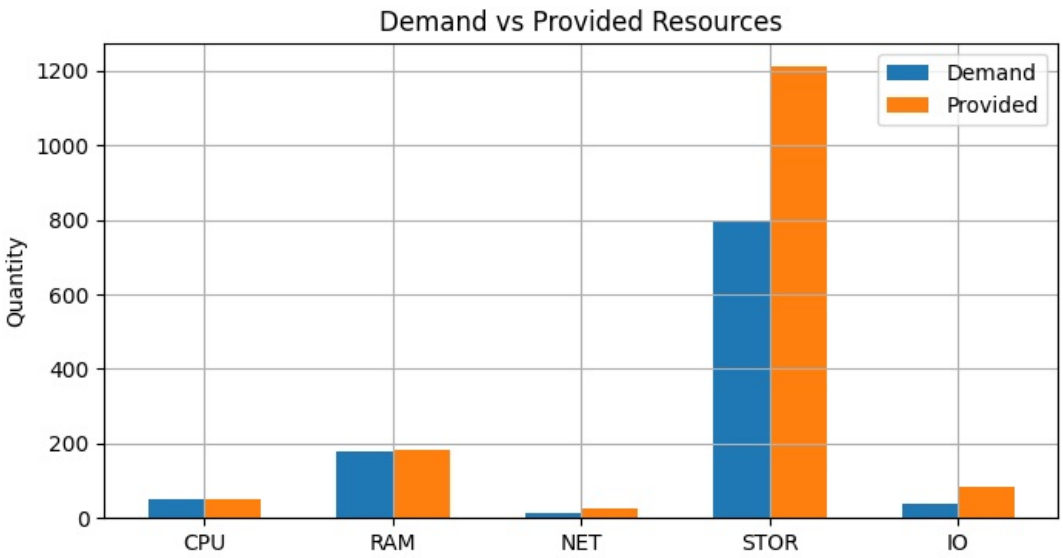



In [9]:

```
# Visual check: demand vs provided resources (after final allocation)
provided = A.T @ final_x  # provided amount for each resource

plt.figure(figsize=(8,4))
inds = np.arange(len(D))
plt.bar(inds - 0.15, D, width=0.3)              # demand
plt.bar(inds + 0.15, provided, width=0.3)       # provided
plt.xticks(inds, ["CPU","RAM","NET","STOR","IO"])
```

```
plt.ylabel("Quantity")
plt.title("Demand vs Provided Resources")
plt.legend(["Demand", "Provided"])
plt.grid(True)
plt.show()
```




In [10]:
```
# Shortfall numbers and quick suggestion if any shortfall remains
shortfall = np.maximum(0, D - provided)
print("Shortfall per resource:", shortfall)
if np.all(shortfall < eps):
    print("All demands satisfied within tolerance.")
else:
    print("Some demands unmet. Consider increasing max_outer, beta, or running more inner iterations.")
```

```
Shortfall per resource: [0. 0. 0. 0. 0.]
All demands satisfied within tolerance.
```

In [ ]: